

就是危险，总会带来“过早结束程序”或“发生不明确行为”的风险。本例要说的是，由客户自己调用 `close` 并不会对他们带来负担，而是给他们一个处理错误的机会，否则他们没机会响应。如果他们不认为这个机会有用（或许他们坚信不会有错误发生），可以忽略它，倚赖 `DBConn` 析构函数去调用 `close`。如果真有错误发生——如果 `close` 的确抛出异常——而且 `DBConn` 吞下该异常或结束程序，客户没有立场抱怨，毕竟他们曾有机会第一手处理问题，而他们选择了放弃。

请记住

- 析构函数绝对不要吐出异常。如果一个被析构函数调用的函数可能抛出异常，析构函数应该捕捉任何异常，然后吞下它们（不传播）或结束程序。
- 如果客户需要对某个操作函数运行期间抛出的异常做出反应，那么 `class` 应该提供一个普通函数（而非在析构函数中）执行该操作。

条款 09：绝不在构造和析构过程中调用 virtual 函数

Never call virtual functions during construction or destruction.

本条款开始前我要先阐述重点：你不该在构造函数和析构函数期间调用 `virtual` 函数，因为这样的调用不会带来你预想的结果，就算有你也不会高兴。如果你同时也是一位 Java 或 C# 程序员，请更加注意本条款，因为这是 C++ 与它们不相同的一个地方。

假设你有个 `class` 继承体系，用来塑模股市交易如买进、卖出的订单等等。这样的交易一定要经过审计，所以每当创建一个交易对象，在审计日志（audit log）中也需要创建一笔适当记录。下面是一个看起来颇为合理的做法：

```
class Transaction {                                //所有交易的 base class
public:
    Transaction( );
    virtual void logTransaction() const = 0;        //做出一份因类型不同而不同
                                                    //的日志记录 (log entry)
    ...
};
```

```
Transaction::Transaction()                //base class 构造函数之实现
{
    ...
    logTransaction();                      //最后动作是忘记这笔交易
}

class BuyTransaction: public Transaction { //derived class
public:
    virtual void logTransaction() const;    //忘记 (log) 此型交易
    ...
};

class SellTransaction: public Transaction { //derived class
public:
    virtual void logTransaction() const;    //忘记 (log) 此型交易
    ...
};
```

现在, 当以下这行被执行, 会发生什么事:

```
BuyTransaction b;
```

无疑地会有一个 `BuyTransaction` 构造函数被调用, 但首先 `Transaction` 构造函数一定会更早被调用; 是的, `derived class` 对象内的 `base class` 成分会在 `derived class` 自身成分被构造之前先构造妥当。`Transaction` 构造函数的最后一行调用 `virtual` 函数 `logTransaction`, 这正是引发惊奇的起点。这时候被调用的 `logTransaction` 是 `Transaction` 内的版本, 不是 `BuyTransaction` 内的版本——即使目前即将建立的对象类型是 `BuyTransaction`。是的, `base class` 构造期间 `virtual` 函数绝不会下降到 `derived classes` 阶层。取而代之的是, 对象的作为就像隶属 `base` 类型一样。非正式的说法或许比较传神: 在 `base class` 构造期间, `virtual` 函数不是 `virtual` 函数。

这一似乎反直觉的行为有个好理由。由于 `base class` 构造函数的执行更早于 `derived class` 构造函数, 当 `base class` 构造函数执行时 `derived class` 的成员变量尚未初始化。如果此期间调用的 `virtual` 函数下降至 `derived classes` 阶层, 要知道 `derived class` 的函数几乎必然取用 `local` 成员变量, 而那些成员变量尚未初始化。这将是一张通往不明确行为和彻夜调试大会串的直达车票。“要求使用对象内部尚未初始化的成分”是危险的代名词, 所以 C++ 不让你走这条路。

其实还有比上述理由更根本的原因: 在 `derived class` 对象的 `base class` 构造期间,

对象的类型是 `base class` 而不是 `derived class`。不只 `virtual` 函数会被编译器解析至 (*resolve to*) `base class`，若使用运行期类型信息 (*runtime type information*，例如 `dynamic_cast` (见条款 27) 和 `typeid`)，也会把对象视为 `base class` 类型。本例之中，当 `Transaction` 构造函数正执行起来打算初始化“`BuyTransaction` 对象内的 `base class` 成分”时，该对象的类型是 `Transaction`。那是每一个 C++ 次成分 (见条款 1) 的态度，而这样的对待是合理的：这个对象内的“`BuyTransaction` 专属成分”尚未被初始化，所以面对它们，最安全的做法就是视它们不存在。对象在 `derived class` 构造函数开始执行前不会成为一个 `derived class` 对象。

相同道理也适用于析构函数。一旦 `derived class` 析构函数开始执行，对象内的 `derived class` 成员变量便呈现未定义值，所以 C++ 视它们仿佛不再存在。进入 `base class` 析构函数后对象就成为一个 `base class` 对象，而 C++ 的任何部分包括 `virtual` 函数、`dynamic_casts` 等等也就那么看待它。

在上述示例中，`Transaction` 构造函数直接调用一个 `virtual` 函数，这很明显而且容易看出违反本条款。由于它很容易被看出来，某些编译器会为此发出一个警告信息 (某些则否，见条款 53 对警告信息的讨论)。即使没有这样的警告，这个问题在执行前也几乎肯定会变得显而易见，因为 `logTransaction` 函数在 `Transaction` 内是个 `pure virtual`。除非它被定义 (不太有希望，但是有可能，见条款 34) 否则程序无法连接，因为连接器找不到必要的 `Transaction::logTransaction` 实现代码。

但是侦测“构造函数或析构函数运行期间是否调用 `virtual` 函数”并不总是这般轻松。如果 `Transaction` 有多个构造函数，每个都需执行某些相同工作，那么避免代码重复的一个优秀做法是把共同的初始化代码 (其中包括对 `logTransaction` 的调用) 放进一个初始化函数如 `init` 内：

```
class Transaction {
public:
    Transaction( )
    { init( ); }                      //调用 non-virtual...
    virtual void logTransaction() const = 0;
    ...
private:
    void init()
    {
        ...
        logTransaction();            //这里调用 virtual!
    }
};
```

这段代码概念上和稍早版本相同，但它比较潜藏并且暗中为害，因为它通常不会引发任何编译器和连接器的抱怨。此时由于 `logTransaction` 是 `Transaction` 内的一个 `pure virtual` 函数，当 `pure virtual` 函数被调用，大多执行系统会中止程序（通常会对此结果发出一个信息）。然而如果 `logTransaction` 是个正常的（也就是 `impure`）`virtual` 函数并在 `Transaction` 内带有一份实现代码，该版本就会被调用，而程序也就会兴高采烈地继续向前行，留下你百思不解为什么建立一个 `derived class` 对象时会调用错误版本的 `logTransaction`。唯一能够避免此问题的做法就是：确定你的构造函数和析构函数都没有（在对象被创建和被销毁期间）调用 `virtual` 函数，而它们调用的所有函数也都服从同一约束。

但你如何确保每次一有 `Transaction` 继承体系上的对象被创建，就会有适当版本的 `logTransaction` 被调用呢？很显然，在 `Transaction` 构造函数(s)内对着对象调用 `virtual` 函数是一种错误做法。

其他方案可以解决这个问题。一种做法是在 `class Transaction` 内将 `logTransaction` 函数改为 `non-virtual`，然后要求 `derived class` 构造函数传递必要信息给 `Transaction` 构造函数，而后那个构造函数便可安全地调用 `non-virtual` `logTransaction`。像这样：

```
class Transaction {
public:
    explicit Transaction(const std::string& logInfo);
    void logTransaction(const std::string& logInfo) const; //如今是个
                                                         //non-virtual 函数
    ...
};

Transaction::Transaction(const std::string& logInfo)
{
    ...
    logTransaction(logInfo); //如今是个
                             //non-virtual 调用
}

class BuyTransaction: public Transaction {
public:
    BuyTransaction( parameters )
        : Transaction(createLogString( parameters )) //将 log 信息
    { ... } //传给 base class 构造函数
    ...
private:
    static std::string createLogString( parameters );
};
```

换句话说由于你无法使用 `virtual` 函数从 `base classes` 向下调用，在构造期间，你可以藉由“令 `derived classes` 将必要的构造信息向上传递至 `base class` 构造函数”替换之而加以弥补。

请注意本例之 `BuyTransaction` 内的 `private static` 函数 `createLogString` 的运用。是的，比起在成员初值列（`member initialization list`）内给予 `base class` 所需数据，利用辅助函数创建一个值传给 `base class` 构造函数往往比较方便（也比较可读）。令此函数为 `static`，也就不可能意外指向“初期未成熟之 `BuyTransaction` 对象内尚未初始化的成员变量”。这很重要，正是因为“那些成员变量处于未定义状态”，所以“在 `base class` 构造和析构期间调用的 `virtual` 函数不可下降至 `derived classes`”。

请记住

- 在构造和析构期间不要调用 `virtual` 函数，因为这类调用从不下降至 `derived class`（比起当前执行构造函数和析构函数的那层）。

条款 10：令 `operator=` 返回一个 *reference to *this*

Have assignment operators return a reference to **this*.

关于赋值，有趣的是你可以把它们写成连锁形式：

```
int x, y, z;  
x = y = z = 15;           //赋值连锁形式
```

同样有趣的是，赋值采用右结合律，所以上述连锁赋值被解析为：

```
x = (y = (z = 15));
```

这里 15 先被赋值给 `z`，然后其结果（更新后的 `z`）再被赋值给 `y`，然后其结果（更新后的 `y`）再被赋值给 `x`。

为了实现“连锁赋值”，赋值操作符必须返回一个 `reference` 指向操作符的左侧实参。这是你为 `classes` 实现赋值操作符时应该遵循的协议：

```
class Widget {  
public:  
    ...
```

```

Widget& operator=(const Widget& rhs)           //返回类型是个 reference,
{                                              // 指向当前对象。
    ...
    return* this;                             //返回左侧对象
}
...
};

```

这个协议不仅适用于以上的标准赋值形式，也适用于所有赋值相关运算，例如：

```

class Widget {
public:
    ...
    Widget& operator+=(const Widget& rhs)       //这个协议适用于
    {                                           // +=, -=, *=, 等等。
        ...
        return *this;
    }
    Widget& operator=(int rhs)                 //此函数也适用，即使
    {                                           // 此一操作符的参数类型
        ...                                   // 不符协定。
        return *this;
    }
    ...
};

```

注意，这只是个协议，并无强制性。如果不遵循它，代码一样可通过编译。然而这份协议被所有内置类型和标准程序库提供的类型如 `string`, `vector`, `complex`, `tr1::shared_ptr` 或即将提供的类型（见条款 54）共同遵守。因此除非你有一个标新立异的好理由，不然还是随众吧。

请记住

- 令赋值 (*assignment*) 操作符返回一个 `reference to *this`。

条款 11: 在 operator= 中处理 “自我赋值”

Handle assignment to self in operator=.

“自我赋值”发生在对象被赋值给自己时：

```

class Widget { ... };
Widget w;
...
w = w; //赋值给自己

```

这看起来有点愚蠢，但它合法，所以不要认定客户绝不会那么做。此外赋值动